

Task 14

Task: AWS Account-Level Cost Spike Detection Automation

Objective

Build an automated solution that can **detect which AWS service is causing high cost at the account level, without any manual checking**, and **notify the team automatically** with relevant details.

Problem Statement

“The AWS monthly bill increased suddenly.

We need an automated way to detect:

- Which AWS service caused the cost spike
- How much the cost increased
- When the increase started
- Notify the team automatically”

Architecture

AWS Cost Explorer



AWS Lambda (Python)



Cost Comparison Logic



Detect Cost Increase per Service



SNS / Email Notification

Task Requirements

1. Enable Cost Explorer

- Enable **AWS Cost Explorer** from the billing settings
- Ensure **daily cost granularity** is available

2. Lambda Automation (Core Task)

Create a **Lambda function (Python)** that:

- Pulls **cost data grouped by AWS service**
- Compares:
 - Today vs Yesterday**OR**
 - Last 7 days vs Previous 7 days
- Identifies:
 - AWS service name
 - Cost difference
 - Percentage increase
- Flags services where:
 - Cost increased by more than **20%****OR**
 - Cost increased by more than **₹500 / \$5** (configurable threshold)

3. Notification

- Configure **SNS email notifications**
- Alert message must clearly include:
 - AWS service name
 - Previous cost
 - Current cost
 - Percentage increase
 - Action required

Example Alert:

 AWS Cost Spike Detected

Service: Amazon EC2

Previous Cost: \$12.40

Current Cost: \$38.20

Increase: +208%

Action Required

4. Scheduling

- Trigger the Lambda function **once per day**
- Use **Amazon Event Bridge**
- Manual execution is **not allowed**

Test Scenario (Mandatory)

To validate your solution:

- Create a temporary cost spike by:
 - Launching an EC2 instance, or
 - Enabling a NAT Gateway, or
 - Creating an Application Load Balancer
- Wait for cost data to appear
- Verify that the alert is triggered correctly

Deliverables

Lambda Python code

- IAM policy used by the Lambda function
- Screenshot of SNS alert email
- Explanation of the cost comparison logic
- Cost optimization suggestion for the detected service

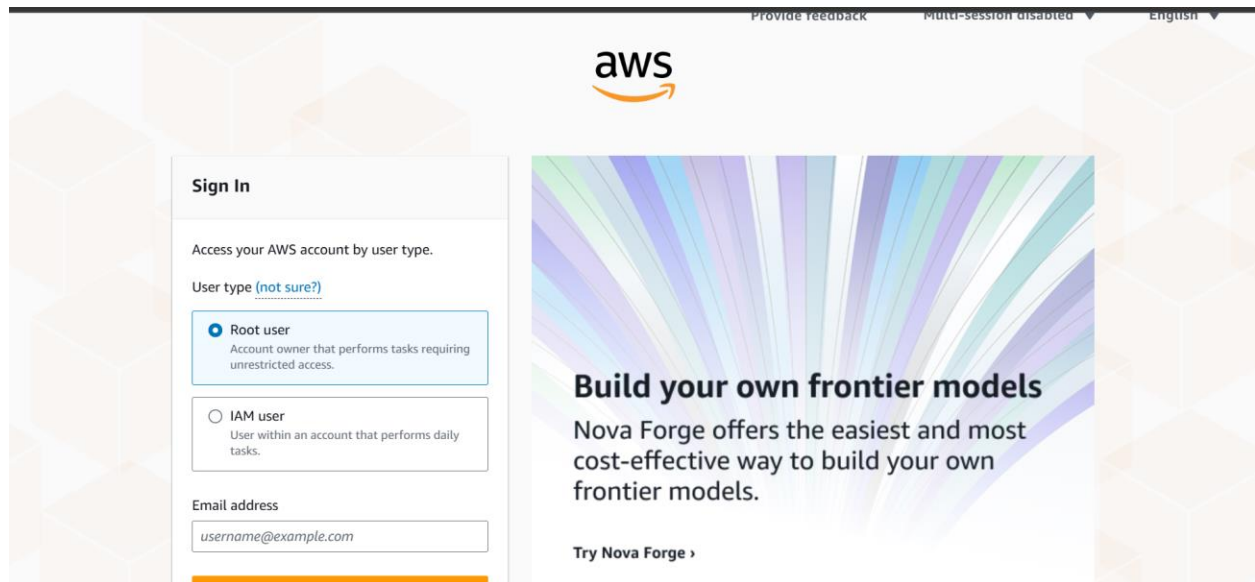
This task will help you understand **AWS billing internals, FinOps concepts, and real-world automation practices** used in production cloud environments.

Solution :

1. Login to your AWS account with credentials and required verification.

(<https://signin.aws.amazon.com/>)

My account – AWS Management Console- Sign in



Before moving forward make sure your **Cost Explorer** is **Enabled** , Sometimes it may take 6-24 hours to expose bill.

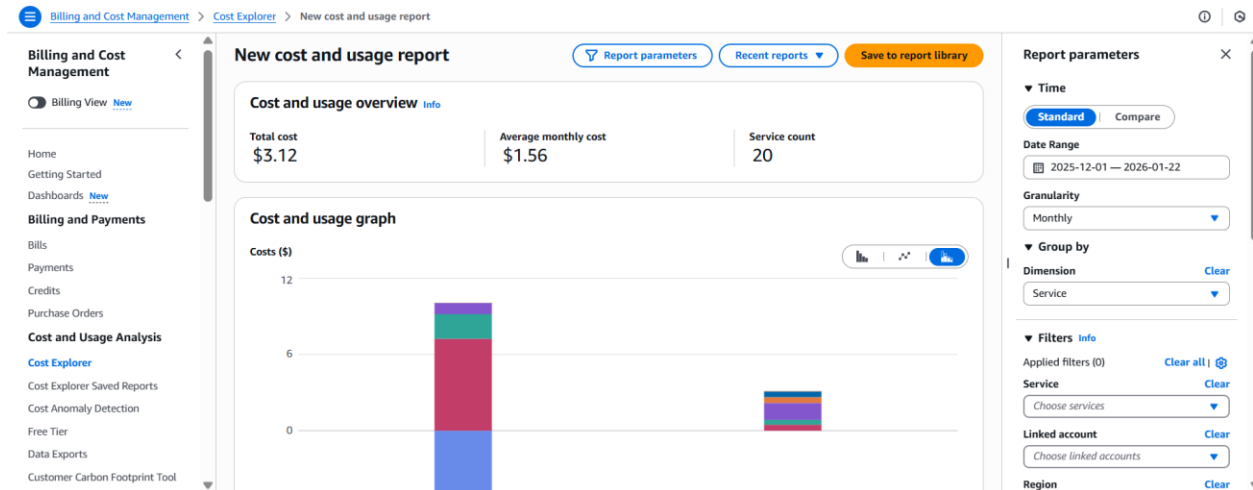
- Go to Billing - Cost Explorer
- Click Enable Cost Explorer
- Wait 12–24 hours for data availability

If it's already Enabled , well and good

2. Enable AWS Cost Explorer (most important)

- Login as root account (important)
- Go to Billing - Cost Explorer

In my account it is already enabled



3. Create SNS Email Notification

This will give updated date and alerts about bills

- Go to SNS - Topics
- Click Create topic
- Type: Standard
- Name : *aws-cost-spike-alerts*

3.1 Create Email Subscription

- Open the topic
- Click Create subscription
- Protocol: Email
- Endpoint: your email address
- Confirm the email (very important)

Amazon SNS > Topics > aws-cost-spike-alerts

Amazon SNS

- Dashboard
- Topics
- Subscriptions
- ▼ Mobile
 - Push notifications
 - Text messaging (SMS)

Details

Name: aws-cost-spike-alerts

ARN: arn:aws:sns:us-east-1:798974631771:aws-cost-spike-alerts

Display name: -

Type: Standard

Topic owner: 798974631771

Subscriptions | Access policy | Data protection policy | Delivery policy (HTTP/S) | Delivery status logging | Encryption | Tags | Integrations

Subscriptions (0)

Search:

Buttons: Edit, Delete, Request confirmation, Confirm subscription, **Create subscription**

ID	Endpoint	Status	Protocol
No subscriptions found You don't have any subscriptions to this topic.			

[Create subscription](#)

Amazon SNS > Subscriptions > Create subscription

Create subscription

Details

Topic ARN:

Protocol:

Endpoint:

After your subscription is created, you must confirm it. [Info](#)

► **Subscription filter policy - optional** [Info](#)
This policy filters the messages that a subscriber receives.

► **Redrive policy (dead-letter queue) - optional** [Info](#)
Send undeliverable messages to a dead letter queue.

AN AWS Notifications <no-reply@sns.amazonaws.com>

To: Ajit Jadhav

Thu 1/22/2026 11:19 AM

You have chosen to subscribe to the topic:

To confirm this subscription, click or visit the link below (If this was in error no action is necessary):

[Confirm subscription](#)

Please do not reply directly to this email. If you wish to remove yourself from receiving all future SNS subscription confirmation requests please send an email to [sns-opt-out](#)

[Reply](#) [Forward](#)

Back to Subscription Now its confirmed.

4. Create IAM Role for Lambda

- Go to IAM - Roles
- Click Create role

- Trusted entity: AWS service
- Service: Lambda

Attach Permissions Policy

Create a inline Policy

The screenshot shows the AWS IAM console interface for creating a role. It is divided into three main steps:

Policy editor

The 'Policy editor' tab is active, showing a JSON policy document with three statements:

```

1 {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Effect": "Allow",
6       "Action": [
7         "ce:GetCostAndUsage"
8       ],
9       "Resource": "*"
10    },
11    {
12      "Effect": "Allow",
13      "Action": [
14        "sns:Publish"
15      ],
16      "Resource": "*"
17    },
18    {
19      "Effect": "Allow",
20      "Action": [
21        "logs:CreateLogGroup",
22        "logs:CreateLogStream",
23        "logs:PutLogEvents"
24      ],
25      "Resource": "*"
26    }
27  ]
28 }

```

On the right, the 'Edit statement' panel is visible, showing a 'Select a statement' section with a '+ Add new statement' button.

Step 1: Select trusted entities

The 'Trust policy' section shows a JSON document for trusting the AWS Lambda service:

```

1 {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Effect": "Allow",
6       "Action": [
7         "sts:AssumeRole"
8       ],
9       "Principal": {
10        "Service": [
11          "lambda.amazonaws.com"
12        ]
13      }
14    }
15  ]
16 }

```

Step 2: Add permissions

The 'Permissions policy summary' table shows the attached policy:

Policy name	Type	Attached as
LambdaCostSpikeRole-policy	Customer managed	Permissions policy

Step 3: Add tags

This section is currently empty.

Create role with this attached policy

5. Create Lambda Function (main Function For task)

- Go to Lambda - Create function
- Author from scratch
- Name: *aws-cost-spike-detector*

- Runtime: Python 3.10
- Execution role:
 - Select *LambdaCostSpikeRole*
- Enter Below Code

```

1  import boto3
2  from datetime import datetime, timedelta
3
4  ce = boto3.client('ce')
5  sns = boto3.client('sns')
6
7  SNS_TOPIC_ARN = "arn:aws:sns:us-east-1:798974631771:aws-cost-spike-alerts"
8
9  PERCENT_THRESHOLD = 20
10 COST_THRESHOLD = 5.0 # USD
11
12 def get_cost(start, end):
13     response = ce.get_cost_and_usage(
14         TimePeriod={
15             'Start': start,
16             'End': end
17         },
18         Granularity='DAILY',
19         Metrics=['UnblendedCost'],
20         GroupBy=[
21             {
22                 'Type': 'DIMENSION',
23                 'Key': 'SERVICE'
24             }
25         ]
26     )
27     return response['ResultsByTime']
28
29 def lambda_handler(event, context):
30     today = datetime.utcnow().date()
31     last_7_start = (today - timedelta(days=7)).strftime('%Y-%m-%d')
32     last_7_end = today.strftime('%Y-%m-%d')
33
34     prev_7_start = (today - timedelta(days=14)).strftime('%Y-%m-%d')
35     prev_7_end = (today - timedelta(days=7)).strftime('%Y-%m-%d')

```

```

12 def get_cost(start, end):
13     response = ce.get_cost_and_usage(
14         TimePeriod={
15             'Start': start,
16             'End': end
17         },
18         Granularity='DAILY',
19         Metrics=['UnblendedCost'],
20         GroupBy=[
21             {
22                 'Type': 'DIMENSION',
23                 'Key': 'SERVICE'
24             }
25         ]
26     )
27     return response['ResultsByTime']
28
29 def lambda_handler(event, context):
30     today = datetime.utcnow().date()
31     last_7_start = (today - timedelta(days=7)).strftime('%Y-%m-%d')
32     last_7_end = today.strftime('%Y-%m-%d')
33
34     prev_7_start = (today - timedelta(days=14)).strftime('%Y-%m-%d')
35     prev_7_end = (today - timedelta(days=7)).strftime('%Y-%m-%d')
36
37     last_7_data = get_cost(last_7_start, last_7_end)
38     prev_7_data = get_cost(prev_7_start, prev_7_end)
39
40     def aggregate(data):
41         costs = {}
42         for day in data:
43             for group in day['Groups']:
44                 service = group['Keys'][0]
45                 amount = float(group['Metrics']['UnblendedCost']['Amount'])
46                 costs[service] = costs.get(service, 0) + amount
47         return costs
48
49     last_7_costs = aggregate(last_7_data)
50     prev_7_costs = aggregate(prev_7_data)
51
52     alerts = []
53
54     for service, current_cost in last_7_costs.items():
55         previous_cost = prev_7_costs.get(service, 0)
56         if previous_cost == 0:
57             continue
58
59         diff = current_cost - previous_cost
60         percent = (diff / previous_cost) * 100
61

```



```

29 def lambda_handler(event, context):
30     alerts = []
31
32     for service, current_cost in last_7_costs.items():
33         previous_cost = prev_7_costs.get(service, 0)
34         if previous_cost == 0:
35             continue
36
37         diff = current_cost - previous_cost
38         percent = (diff / previous_cost) * 100
39
40         if diff > COST_THRESHOLD or percent > PERCENT_THRESHOLD:
41             alerts.append(
42                 f"""
43 Service: {service}
44 Previous Cost: ${previous_cost:.2f}
45 Current Cost: ${current_cost:.2f}
46 Increase: (percent:.2f)%
47 """
48             )
49
50     if alerts:
51         message = " 🚨 AWS Cost Spike Detected\n\n" + "\n".join(alerts) + "\nAction Required"
52         sns.publish(
53             TopicArn=SNSTOPIC_ARN,
54             Subject="AWS Cost Spike Alert",
55             Message=message
56         )
57
58     return {"status": "completed"}
59
60 Amazon Q Tip 1/3: Start typing to get suggestions. ([ESC] to exit)

```

Deploy

Successfully updated the function aws-cost-spike-detector.

EXPLORER

- aws-cost-spike-detector
 - lambda_function.py

DEPLOY ✓ Current

Deploy (Ctrl+Shift+U)

Test (Ctrl+Shift+I)

TEST EVENTS (NONE SELECTED)

+ Create new test event

```

3
4 ce = boto3.client('ce')
5 sns = boto3.client('sns')
6
7 SNSTOPIC_ARN = "arn:aws:sns:us-east-1:798974631771:aws-cost-spike-alerts"
8
9 PERCENT_THRESHOLD = 20
10 COST_THRESHOLD = 5.0 # USD
11
12 def get_cost(start, end):
13     response = ce.get_cost_and_usage(
14         TimePeriod={
15             'Start': start,
16             'End': end
17         },
18         Granularity='DAILY',
19         Metrics=['UnblendedCost'],
20         GroupBy=[
21             {
22                 'Type': 'DIMENSION',
23                 'Key': 'SERVICE'
24             }
25         ]
26     )
27     return response['ResultsByTime']
28
29 def lambda_handler(event, context):

```

6. Schedule Lambda Via EventBridge

- Go to EventBridge - Rules
- Create rule
- Name: daily-cost-spike-check
- Rule Type : Schedule

Amazon EventBridge

A serverless service for building event-driven applications

Amazon EventBridge is a serverless service that uses events to connect application components together, making it easier for developers to build scalable event-driven applications.

Get started

- ☐ **EventBridge Rule with event pattern**
A rule matches incoming events and sends them to targets for processing.
- ☐ **EventBridge Scheduled rule**
A rule that will invoke a target at a scheduled time.
- ☐ **EventBridge Pipe**
A pipe connects an event source to a target with optional filtering and enrichment.
- ☒ **EventBridge Schedule**
A schedule invokes a target one-time or at regular intervals defined by a cron or rate expression.
- ☐ **EventBridge Schema registry**
Schema registries collect and organize schemas.

Create schedule

Pricing

- Step 1 **Specify schedule detail**
- Step 2 Select target
- Step 3 Settings
- Step 4 Review and create schedule

Specify schedule detail

Schedule name and description

Schedule name

daily-cost-spike-check

Use only letters, numbers, dashes, dots or underscores. Max 64 characters.

Description - optional

Daily Check cost of resources

Maximum of 512 characters

Schedule group

Each schedule needs to be placed in a schedule group. By default, a schedule is placed in the 'Default' group. You can also [create your own schedule group](#). You can only add tags to a schedule group, not a schedule.

default



Schedule pattern

Occurrence | Info

You can define an one-time or recurrent schedule.

- ☐
- One-time schedule

Study patients

Occurrence | Info

You can define an one-time or recurrent schedule.

- ☐
- One-time schedule

Time zone

The time zone for the schedule.

(UTC+05:30) Asia/Calcutta

Schedule type

Choose the schedule type that best meets your needs.

- ☐ **Cron-based schedule**
A schedule set using a cron expression that runs at a specific time, such as 8:00 a.m. PST on the first
 - ☒ **Rate-based schedule**
A schedule that runs at a regular rate, such as every 10 minutes.

Rate expression [Info](#)

Enter a value and the unit of time to run the schedule.

rate (1 days)

Flexible time window

If you choose a flexible time window, Scheduler invokes your schedule within the time window you specify. For example, if you choose 15 minutes, your schedule runs within 15 minutes after the schedule start time.

Select

Amazon EventBridge > Schedules > Create schedule

● Review and create schedule

CodeBuild
StartBuild

CodePipeline
StartPipelineExecution

Amazon ECS
RunTask

Amazon EventBridge
PutEvents

Amazon Inspector V1
StartAssessmentRun

Kinesis Data Firehose
PutRecord

Kinesis Data Streams
PutRecord

AWS Lambda
Invoke

Amazon SNS
Publish

Amazon SQS
SendMessage

SageMaker
StartPipelineExecution

AWS Step Functions
StartExecution

Invoke

AWS Lambda

Lambda function

aws-cost-spike-detector

► Configure version/alias

Universal target definition

Create new Lambda function

Amazon EventBridge > Schedules > Create schedule

Enabled

Action after schedule completion

Amazon_EventBridge_Scheduler_LAMBDA_2b2afe6f2f

Retry policy and dead-letter queue (DLQ)

Retry policy

Max age of event: -

Dead-letter queue ARN

None

Retry policy

Maximum retries: -

Encryption

Customer master key (CMK)

aws/scheduler

Description

Default master key that protects my Amazon EventBridge Scheduler data when no other key is defined

Key ARN

-

Cancel Previous Create schedule

7. Test

for this we must create cost spike

Choose one only:

- Launch EC2
- Create NAT Gateway

So, Launching Ec2 instance for create Cost Spike

The first screenshot shows the 'Launch an instance' wizard. The 'Name and tags' section has 'testing-Cost-Spike' entered. The 'Application and OS Images' section shows 'Amazon Linux' selected under 'Quick Start'. The 'Summary' panel on the right shows 1 instance, Amazon Linux 2023 AMI, t3.micro instance type, new security group, and 1 volume (8 GiB).

The second screenshot shows the 'Instance configuration' step. The 'Instance type' is 't3.micro'. The 'Key pair' section shows 'Proceed without a key pair'. The 'Network settings' section shows 'vpc-02486129ea4c880ca' selected. The 'Summary' panel on the right is identical to the first screenshot.

The third screenshot shows the 'Instances' list in the AWS Management Console. The instance 'testing-Cost-Spike' with ID 'i-0bc34ba60e8a1b393' is shown in a 'Running' state. The table below shows the instance details:

Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4 DNS
testing-Cost-Spike	i-0bc34ba60e8a1b393	Running	t3.micro	Initializing	View alarms +	us-east-1f	ec2-18-207-254-87.co...

Now we will keep instance running for increase cost spike

And wait for alert



So we got Alert for our resources

We can ensure controlling AWS costs using automation instead of reacting after huge bills.

Purpose: to learn and prove real-world cloud engineering and FinOps skills.
